COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010/Data Structures and Algorithm	**SECTION:**	CPE21S3
DATE PERFORMED:	July 28, 2026	**DATE SUBMITTED:**	August 1, 2026
NAME:	Gresola, Aryanna Keith G.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 4.1 **Array Based vs. STL Stack Implementation**

1. PROCEDURE OUTPUT

ILO A: Implement a stack using the C++ STL (std::stack).

INPUT

ILOA > main.cpp > main()

```cpp
#include <iostream>
#include <stack> // call the stack from the STL

int main(){
    std::stack<int> newStack; // created an object

    // testing the empty() function to check if the stack is empty
    std::cout << "Is the stack empty? " << newStack.empty() << std::endl;

    //insert a 3 elements:
    newStack.push(3);
    std::cout << "Top of stack: " << newStack.top() << std::endl;
    newStack.push(2);
    std::cout << "Top of stack: " << newStack.top() << std::endl;
    newStack.push(1);
    std::cout << "Top of stack: " << newStack.top() << std::endl;

    // checking if the stack is empty
    std::cout << "Is the stack empty? " << newStack.empty() << std::endl;

    newStack.push(6);
    newStack.push(7);
    newStack.push(8);
    // checking the size of the current stack
    std::cout << "Size of the stack is: " << newStack.size() << std::endl;
```

OUTPUT

```
Is the stack empty? 1
Top of stack: 3
Top of stack: 2
Top of stack: 1
Is the stack empty? 0
Size of the stack is: 6
```

ANALYSIS

This part of the program begins by checking whether the newly created stack is empty. Since the stack has just been initialized and does not contain any data, the empty() function returns true, indicating that the stack is empty. Next, the newStack.push() function is used to insert elements into the stack. After inserting the element, the program checks again if the stack is empty. This time, the empty() function returns false because the stack now contains an elemenst. The program then continues to insert additional elements into the stack using the same push() function. After a total of six elements have been pushed onto the stack, the size() function is called to determine the number of elements currently stored. Since six elements have been inserted, the program displays **6** in the console.

INPUT

```
26
27      // emplace -> construct and inert element -> public member function
28      std::cout << "EMPLACE" << std::endl;
29      newStack.emplace(9);
30      std::cout << "New top of the stack after emplace: " << newStack.top() << std::endl;
31
32      // swap -> swap the contents of the public member functions
33      std::cout << "SWAP" << std::endl;
34      std::stack<int> secondStack;
35
36      secondStack.push(10);
37      secondStack.push(11);
38      secondStack.push(12);
39
40      std::cout << "Before swap: " << std::endl;
41      std::cout << "Top of newStack: " << newStack.top() << std::endl;
42      std::cout << "Top of secondStack: " << secondStack.top() << std::endl;
43
44      newStack.swap(secondStack);
45
46      std::cout << "After swap: " << std::endl;
47      std::cout << "Top of newStack: " << newStack.top() << std::endl;
48      std::cout << "Top of secondStack: " << secondStack.top() << std::endl;
49
50
51      return 0;
52  }
```

OUTPUT

```
 EMPLACE
New top of the stack after emplace: 9

 SWAP
Before swap:
Top of newStack: 9
Top of secondStack: 12
After swap:
Top of newStack: 12
Top of secondStack: 9
```

ANALYSIS

This part of the program demonstrates the use of the emplace() and swap() functions. The program first uses the emplace() function. Unlike push(), which inserts an already existing object into the stack, emplace() constructs a new object directly in the stack. The newly created element is placed at the top of the stack, making it the new top element.

Next, the program demonstrates the swap() function. To better illustrate its operation, another stack containing a different set of data is created. When the swap() function is executed, it exchanges the contents of the two stacks. To verify that the swap was successful, the program displays the top element of each stack using the top() function. The output shows that the top elements now belong to the opposite stacks, confirming that the swap operation was performed successfully.

ILO B.1: Implement a stack using a Static Array.

INPUT

ILOB > main.cpp > main()

```cpp
#include <iostream>

const size_t maxCap = 100;
int stack[maxCap];
int top = -1, newData, elements;

// prototyping
bool isFull();
bool isEmpty();
void push();
void pop();
void Top();
void displayAll();
```

```cpp

int main(){
    int choice;
    // asked the user for the maximum elements to be inserted
    std::cout << "Enter the number of maximum elements for the new stack (max: 100): ";
    std::cin >> elements;

    while(true){
        std::cout << "STACK OPERATIONS: " << std::endl;
        std::cout << " 1. Push\n 2. Pop\n 3. Top\n 4. isEmpty\n 5. isFull\n 6. Display All\n";
        std::cout << "Choose Operation: ";
        std::cin >> choice;

        switch(choice){
            case 1: push();
            break;
            case 2: pop();
            break;
            case 3: Top();
            break;
            case 4: std::cout << isEmpty() << std::endl;
            break;
            case 5: std::cout << isFull() << std::endl;
            case 6: displayAll();
            break;
            default: std::cout << "Invalid choice." << std::endl;
            break;

        }
    }

    return 0;
}
```

```cpp

bool isEmpty(){
    // top is -1
    if(top == -1){
        std::cout << "Stack Underflow" << std::endl;
        return true;
    }
    return false;
}

bool isFull(){
    // not equal to the maxCap
    if(top >= elements-1){
        std::cout << "Stack overflow.";
        return true;
    }
    return false;
}
```

```
66
67  void push(){
68      // check if the stack is full. if yes return an error
69      if(isFull()) return;
70      // ask the use for a value to be pushed into the stack
71      std::cout << "What number you want to add to the stack? ";
72      std::cin >> newData;
73      //store inside the stack? check pre incre vs. post incre
74      stack[++top] = newData;
75  }
76
77  void pop(){
78      // check if there are no more elements inside the stack
79      if(isEmpty()) return;
80
81      //display the element to be popped:
82      std::cout << "Popping: " << stack[top] << std::endl;
83
84      //decrement the top
85      top--;
86  }
87
88  void Top(){
89      // error checking
90      if(isEmpty()) return;
91
92      //display the top element
93      std::cout << "The top element is: " << stack[top] << std::endl;
94  }
95
96  void displayAll(){
97      if (isEmpty()) return;
98
99      std::cout << "Contents of the Stack: " << std::endl;
100
101     for (int i = top; i >= 0; i--){
102         std::cout << stack[i] << std::endl;
103     }
104 }
```

OUTPUT

```
Enter the number of maximum elements for the new stack (max: 100): 5
STACK OPERATIONS:
 1. Push
 2. Pop
 3. Top
 4. isEmpty
 5. isFull
 6. Display All
Choose Operation:
```

ANALYSIS

This part of the program displays the main menu, which contains all the available functions that the user can perform within the program.

OUTPUT (PUSH)

```
STACK OPERATIONS:
 1. Push
 2. Pop
 3. Top
 4. isEmpty
 5. isFull
 6. Display All
Choose Operation: 1
What number you want to add to the stack? 5
```

ANALYSIS

This part of the program demonstrates the push() function. This function allows the user to insert new data into the stack, serving as the Add feature of the program. Each new element is placed at the top of the stack, following the Last-In, First-Out (LIFO) principle.

OUTPUT (POP)

```
STACK OPERATIONS:
 1. Push
 2. Pop
 3. Top
 4. isEmpty
 5. isFull
 6. Display All
Choose Operation: 2
Popping: 0
```

ANALYSIS

The second feature of the program is the pop() function. It removes the top element of the stack, which is the most recently added data. This function serves as the Delete feature of the program.

OUTPUT (TOP)

```
STACK OPERATIONS:
 1. Push
 2. Pop
 3. Top
 4. isEmpty
 5. isFull
 6. Display All
Choose Operation: 3
The top element is: 2
```

ANALYSIS

This part of the output demonstrates the top() function. It displays the data currently at the top of the stack without modifying or removing any elements. This function allows the user to view the most recently added item.

OUTPUT (ISEMPTY)

```
STACK OPERATIONS:
 1. Push
 2. Pop
 3. Top
 4. isEmpty
 5. isFull
 6. Display All
Choose Operation: 4
0
```

(with contents)

```
STACK OPERATIONS:
 1. Push
 2. Pop
 3. Top
 4. isEmpty
 5. isFull
 6. Display All
Choose Operation: 4
Stack Underflow
1
```

(with no contents)

ANALYSIS

This part of the program demonstrates the isEmpty() function. This function checks whether the stack contains any data. It returns 0 (False) if the stack is not empty, indicating that it contains one or more elements. If the stack is empty, it returns 1 (True) and displays the message "Stack Underflow" to indicate that there are no elements in the stack.

OUTPUT(ISFULL)

```
STACK OPERATIONS:
 1. Push
 2. Pop
 3. Top
 4. isEmpty
 5. isFull
 6. Display All
Choose Operation: 5
0
```

```
STACK OPERATIONS:
 1. Push
 2. Pop
 3. Top
 4. isEmpty
 5. isFull
 6. Display All
Choose Operation: 5
Stack overflow.1
```

ANALYSIS

This part of the program demonstrates the isFull() function. This function checks whether the stack has reached its maximum capacity. If the stack is full, it displays the message "Stack Overflow", indicating that no more elements can be added. If the stack has no elements, it displays "Stack Underflow", indicating that the stack is empty.

OUTPUT(DISPLAYALL)

```
STACK OPERATIONS:
 1. Push
 2. Pop
 3. Top
 4. isEmpty
 5. isFull
 6. Display All
Choose Operation: 6
Contents of the Stack:
1
2
3
```

ANALYSIS

This part of the program demonstrates the displayAll() function. This function displays all the elements currently stored in the stack. It prints every item in the stack, allowing the user to view all of its contents.

B.2. Stacks using Linked Lists

INPUT (LISTSTACK.H)

ILOB2 > C listStack.h > displayAll<T>()

```
1   #ifndef LIST_STACK_H
2   #define LIST_STACK_H
3   #include <iostream>
4
5   //class mode with data and next members
6   template <typename T>
7   class Node{
8       public:
9       T data;
10      Node<T>* next = nullptr;
11  };
12
13  // global
14  template <typename T>
15  Node<T>* head = nullptr;
```

```cpp

// pushing operation:
template <typename T>
void push(T newData){

    // create a new node
    Node<T>* newNode = new Node<T>;
    // assign the data to the new node
    newNode->data = newData;
    // point the newnode next to the current top
    newNode->next = head<T>;
    // assign the new top to the new node
    head<T> = newNode;
}

//isEmpty operation:
template <typename T>
bool isEmpty(){
    return head<T> == nullptr;
}

// popping operation:
template <typename T>
T pop(){
    // if the stack is empty error catch:
    if(isEmpty<T>()){
        std::cout << "Stack underflow." << std::endl;
        return T();
    }
    // saving a current top node to a temporary variable
    Node<T>* temp = head<T>;
    // store the data to a temporary variable
    T tempVal = temp->data;
    // move the head to next node
    head<T> = head<T>->next;
    // free up the memory
    delete temp;
    //return the temporary variable data
    return tempVal;
}
```

```
56
57  // top operation:
58  template <typename T>
59  void Top(){
60      // what if the stack is empty?
61      if(isEmpty<T>())
62          return;
63      std::cout << "The top is: " << head<T>->data << std::endl;
64  }
65
66  // displaying all element in stack
67  template <typename T>
68  void displayAll(){
69      // first check if the stack is empty
70      if (isEmpty<T>()){
71          std::cout << "The stack is empty." << std::endl;
72          return;
73      }
74      // starts at the top (head)
75      Node<T>* current = head<T>;
76      std::cout << "Contents of Stack: ";
77      // keeps moving til there are no more nodes
78      while (current != nullptr){
79          // prints the current node data
80          std::cout << current->data << " ";
81          // move to the next node til null
82          current = current->next;
83      }
84      std::cout << std::endl;
85  }
86
87
88  #endif
```

INPUT (MAIN.CPP)

ILOB2 > main.cpp > ...

```cpp
#include <iostream>
#include"listStack.h"

int main(){
    std::cout << " ------ PUSHING 'I' 'E' 'E' 'E' ------ \n";

    // pushing
    push('I');
    push('E');
    push('E');
    push('E');

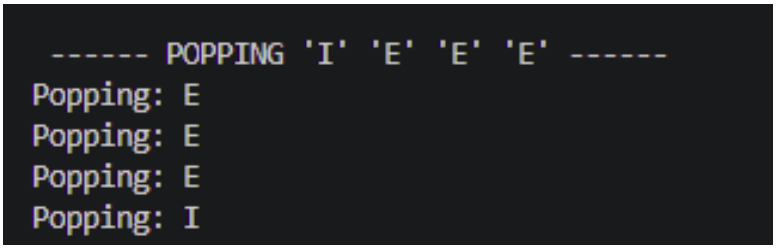
    displayAll<char>();
    // display top
    Top<char>();
    std::cout << std::endl;

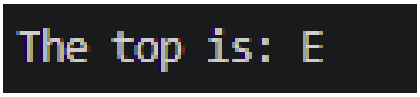
    // pop and print each element
    std::cout << " ------ POPPING 'I' 'E' 'E' 'E' ------ \n";
    while(!isEmpty<char>()){
        std::cout << "Popping: " << pop<char>() << std::endl;
    }
    // test stack underflow
    pop<char>();

    return 0;
}
```

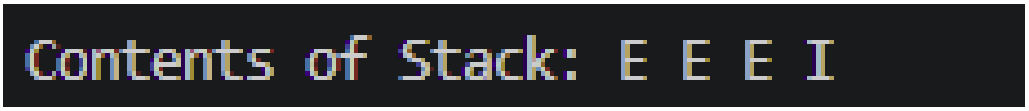
OUTPUT (PUSH)

DESCRIPTION
This function adds a new element to the top of the stack by creating a new node and making it the new top.

OUTPUT (POP)

DESCRIPTION
This function removes the top element from the stack and return its value. If the stack is empty it will display the message “Stack Underflow."

OUTPUT (TOP)

DESCRIPTION
This function displays the element at the top of the stack.

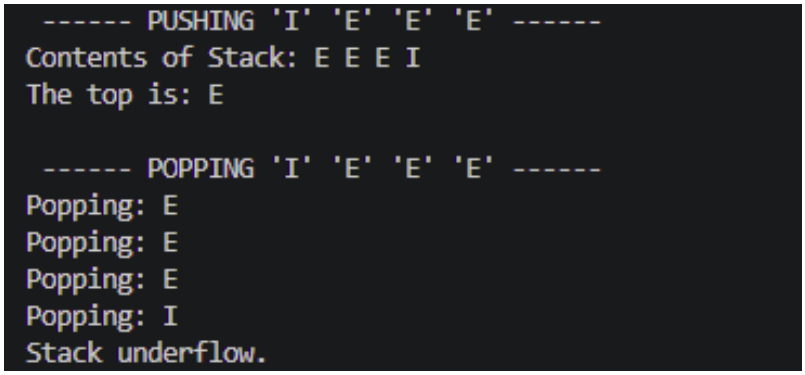
OUTPUT (DISPLAYALL)



DESCRIPTION

This function displays all the elements in the stack from the top to the bottom by going through each node one by one.

OVERALL OUTPUT



2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve problems using an implementation of stack:

a. Stack using Arrays

INPUT (ARRAYSTACK.H)

SUPPLE_A > C arrayStack.h > ...

```
#ifndef ARRAYSTACK_H
#define ARRAYSTACK_H

#include <iostream>

const int MAX = 100;
char stack[MAX];
int top = -1;

// push operation:
void push (char value){
    if(top == MAX - 1)
        return;

    stack[++ top] = value;
}

// popping operation:
char pop(){
    if(top == -1)
        return '\0';

    return stack[top--];
}

// top operation:
char Top(){
    if (top == -1)
        return '\0';
    return stack[top];
}

// isEmpty operation:
bool isEmpty(){
    return top == -1;
}

#endif
```

INPUT (MAIN.CPP)

SUPPLE_A > main.cpp > ...

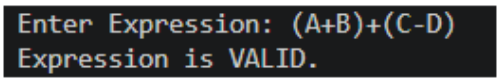
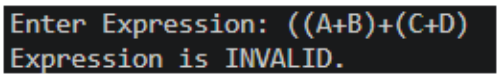
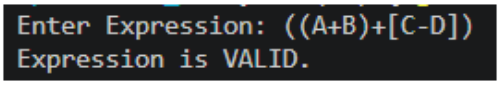
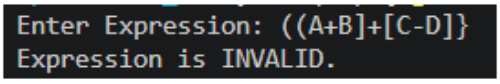
```cpp
#include <iostream>
#include <string>
#include "arrayStack.h"


bool isBalanced(std::string expression){
    top = -1; // resetting stack
    for (char c : expression){

        // opening symbols
        if (c == '(' || c == '[' || c == '{'){
            push(c);
        } else if (c == ')' || c == '}' || c == ']'){ // closing symbols
            if (isEmpty()){
                return false;
            }
            char open = pop();
            if ((open == '(' && c != ')') ||
                (open == '[' && c != ']')  ||
                (open == '{' && c != '}')){
                return false;
            }
        }
    }
    return isEmpty();
}

int main() {
    std::string expression;
    std::cout << "Enter Expression: ";
    getline(std::cin, expression);

    if (isBalanced(expression))
        std::cout << "Expression is VALID." << std::endl;
    else
        std::cout << "Expression is INVALID." << std::endl;
    return 0;
}
```

OUTPUT			
Expression	**Valid? (Y/N)**	**Output**	**Analysis**
(A+B)+(C-D)	**Y**		This expression is valid because every opening symbol is matched with the correct closing symbol. The array stack becomes empty after checking the expression.
((A+B)+(C+D)	**N**		This expression is invalid because one opening parenthesis is left in the array stack. Since the stack is not empty, the expression is not balanced.
((A+B)+[C-D])	**Y**		This expression is valid because all parentheses and brackets are matched correctly. The array stack is empty after the checking process.
((A+B]+[C-D]}	**N**		This expression is invalid because the opening and closing symbols do not match. The array stack detects the mismatch and marks the expression as unbalanced.

ANALYSIS
This program is divided into two parts: the header file (arrayStack.h) and the main program. The header file contains the stack operations such as push(), pop(), Top(), and isEmpty(), where each function performs a specific task in managing the stack. Meanwhile, the main program contains the isBalanced() function, which uses these stack operations to check whether an expression has balanced symbols. The program follows the Last In, First Out (LIFO) principle, where every opening symbol is pushed onto the stack and every closing symbol pops the most recent opening symbol for comparison. If all symbols are matched correctly and the stack is empty after checking the entire expression, the expression is considered valid. Otherwise, if there is a mismatched pair or there are still symbols left in the stack, the expression is considered invalid.

b. Stack using Linked Lists

INPUT (LL_STACK.h)

SUPPLE.B > C LL_STACK.h > ...

```
#ifndef LL_STACK_H
#define LL_STACK_H

#include <iostream>

class Node{
    public:
        char data;
        Node* next = nullptr;
};

Node* head = nullptr;

//push opereation
void push(char newData){
    //create new node
    Node* newNode = new Node;
    //assign data
    newNode->data = newData;
    //point the new node to the current head
    newNode->next = head;
    //assign the new head
    head = newNode;
}

//isEmpty operation
bool isEmpty(){
    return head == nullptr;
}

//pop operation
char pop(){
    //checks if the stack is empty
    if(isEmpty()){
        std::cout << "Stack Underflow." << std::endl;
        return '\0';
    }
```

```
    //save the current top node
    Node* temp = head;
    //save the data
    char tempVal = temp->data;
    //move head to the next node
    head = head->next;
    //free memory
    delete temp;
    return tempVal;
}

//top operation
char Top(){
    if(isEmpty()){
        return '\0';
    }
    return head->data;
}

#endif
```

INPUT (MAIN.CPP)

SUPPLE.B > main.cpp > ...

```cpp
#include <iostream>
#include <string>
#include "LL_STACK.h"

bool isBalanced(std::string expression){
    head = nullptr; //resetting stack

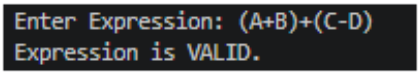
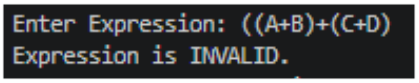
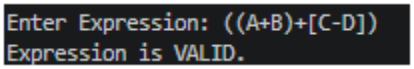
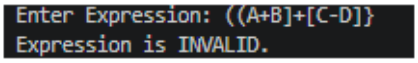
    for(char c : expression){
        //opening symbols
        if(c == '(' || c == '[' || c == '{'){
            push(c);
        }else if(c == ')' || c == ']' || c == '}'){
            if(isEmpty()){
                return false;
            }
            char open = pop();

            if((open == '(' && c != ')') ||
                (open == '[' && c != ']') ||
                (open == '{' && c != '}')){
                    return false;
            }
        }
    }
    return isEmpty();
}

int main(){
    std::string expression;

    std::cout << "Enter Expression: ";
    getline(std::cin, expression);

    if(isBalanced(expression)){
        std::cout << "Expression is VALID." << std::endl;
    }else{
        std::cout << "Expression is INVALID." << std::endl;
    }
    return 0;
}
```

OUTPUT			
Expression	**Valid? (Y/N)**	**Output**	**Analysis**
(A+B)+(C-D)	**Y**		This expression is valid because every opening symbol is matched with the correct closing symbol. The linked list stack becomes empty after checking the expression.
((A+B)+(C+D)	**N**		This expression is invalid because one opening parenthesis remains in the linked list stack. Since the stack is not empty, the expression is not balanced.
((A+B)+[C-D])	**Y**		This expression is valid because all parentheses and brackets are matched correctly. The linked list stack is empty after the checking process.
((A+B]+[C-D]}	**N**		This expression is invalid because the opening and closing symbols do not match. The linked list stack detects the mismatch and marks the expression as unbalanced.

ANALYSIS
This program is divided into two files: the header file (LL_STACK.h) and main.cpp. The header file contains the stack functions such as push(), pop(), Top(), and isEmpty(), where each function is responsible for managing the linked list stack. These functions are then called in main.cpp to perform their specific operations. Meanwhile, the main.cpp file contains the isBalanced() function, which checks whether the symbols in an expression are balanced. The program works by following the Last In, First Out (LIFO) principle, where every opening symbol is pushed into the linked list stack and every closing symbol pops the most recent opening symbol for comparison. If all symbols are matched correctly and the stack is empty after checking the entire expression, the expression is considered valid. Otherwise, the expression is identified as invalid.

OVERALL ANALYSIS

In this supplementary activity, two different implementations of a stack were used: Array Stack and Linked List Stack. Although they are implemented differently, both follow the same logic in checking whether the symbols in an expression are balanced. They both use the Last In, First Out (LIFO) principle, where every opening symbol is pushed into the stack and every closing symbol pops the most recent opening symbol for comparison. The main difference is how the data is stored. The Array Stack stores the symbols in a fixed-size array and uses the top index to keep track of the top element. On the other hand, the Linked List Stack stores the symbols in dynamically allocated nodes connected by pointers and uses the head pointer as the top of the stack. Despite these differences in implementation, both programs perform the same function and produce the same result in determining whether an expression is balanced or not.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.